ROBOTIC SPATIOTEMPORAL EDITING: A UNIFIED INTERFACE FOR CONTROLLABLE ACTION GENERATION VIA MASKED ACTION RECONSTRUCTION

Yanbang Li[1,2,*], **Bu He**[1,*], **Yi Tian**[1], **Yu-Chiao Huang**[1], **Chenyu Zhang**[1,3], **Ruoqu Chen**[1,3], **Yuanzhe Liu**[4], **Yuxiang Gao**[2], **Mengdi Xu**[1,†]

[1]Tsinghua University [2]CocoMatrix [3]Shanghai Qizhi Institute [4]UIUC

[*]Equal contribution [†]Corresponding author

https://robotic-spatiotemporal-editing.github.io/rose.github.io/

ABSTRACT

Generative policies can synthesize robot behaviors, yet controlling and editing them through diverse spatiotemporal references remains challenging. Real-world manipulation requires robots to satisfy user-specified constraints on where, when, and how actions are executed. Our key insight is that spatiotemporal control references, such as keypoints, flows, orientations, and keyposes, can be viewed as partial specifications of the same temporal Cartesian action space. We introduce the **MAP (Masked Action Space)** interface to unify these action references and formulate controllable manipulation as an action inpainting problem. Our **Inpainting Policy (IP)** reconstructs missing action dimensions of the end-effector, combining progress-guided alignment with multi-mask training, supporting unseen spatiotemporal constrains. We further adopt a training-free **RePaint** variant that iteratively inpaints the action during denoising to reduce control error. On ManiSkill and LIBERO-10, our policy achieves average task success rates of 92.67% and 64.67%, respectively. Real-world experiments demonstrate spatiotemporal control with average positional and rotational errors of $44.3\,\mathrm{mm}$ and $7.8°$, respectively.

1 INTRODUCTION

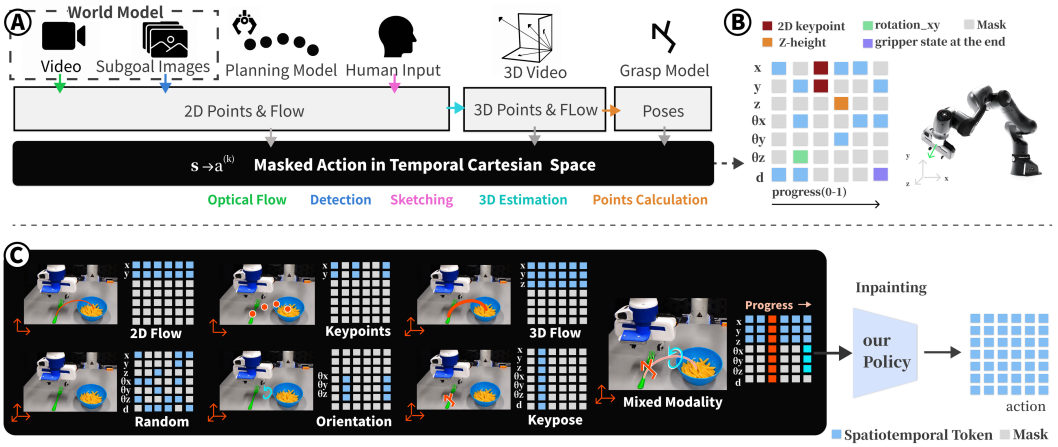


Figure 1: **Unified spatiotemporal control via action inpainting. Top:** To enable unified spatiotemporal control from heterogeneous model-generated and human-provided references, we introduce **Masked Action Space (MAP)**, which represents these references as partial specifications of temporal Cartesian actions. Progress values specify when constraints apply, instead of timestep. **Bottom:** Our **Inpainting Policy (IP)** predicts these masks through action inpainting, supporting potentially unlimited spatiotemporal reference types and unseen reference combinations.

Generative models have recently demonstrated remarkable progress in robotic manipulation, enabling policies to synthesize complex robot behaviors from visual observations and task descriptions (Chi et al., 2025; Black et al., 2026; Brohan et al., 2023). However, generating plausible behaviors alone is insufficient for deploying robots in the physical world. Real-world tasks require robots not only to decide what to do but also to satisfy constraints on **where, when, and how** to execute an action (Rempe et al., 2026). Moreover, humans and intelligent systems often need to refine, modify, and compose robot behaviors by adding spatiotemporal constraints. For example, when tidying a table, a user may use a hand gesture or a sketch instruction (Li et al., 2025b) to specify where an object should be placed, rather than merely asking the robot to tidy the table.

How can we edit robot behaviors in space and time? In image and video generation, the shift from generating content from user instructions to enabling fine-grained, controllable editing has reshaped image and video generation industry. Reference-based interfaces, such as ControlNet (Zhang et al., 2023), enable users to steer synthesis through interpretable signals like edges, depth, and poses. They demonstrate that a well-designed interface can bridge human's spatial intent and generative outcomes.

Meanwhile, robotics is gaining access to diverse spatiotemporal references from world models, 3D/4D videos, advanced agents, grasp models, and human demonstrations (Jang et al., 2025; Gao et al., 2026; Chen et al., 2026b; Murali et al., 2025; Fan et al., 2025b), including keypoints (Huang et al., 2024), flow (Xu et al., 2024), keyposes (Yu et al., 2024), and grasp proposals (Fang et al., 2023; Hao et al., 2025) (see Figure 1(A)). These representations are physically grounded and low-dimensional, but they only provide partial information on future robot motion rather than complete executable actions. Existing approaches typically address this gap with signal-specific policies or conditioning mechanisms. This motivates the need for **a unified action interface** that can transform diverse spatiotemporal references into controllable robot behaviors.

Our key insight is that heterogeneous spatiotemporal control are not different control problems, but partial action tokens of the same underlying Temporal Cartesian Action Space. We design **Masked Action Space (MAP)** as a shared interface that represents these references through partially specified action tokens and a binary mask (see Figure 1(B,C)). We formulate controllable action generation as **a masked action inpainting problem**, and we propose **Inpainting Policy (IP)** to reconstruct masked components in MAP and turn them into a complete executable trajectory (see Section 3.3).

To enable this formulation, we design three components. **(1)** A **progress-guided sliding window** aligns control references with the current task stage using estimated progress value (Chen et al., 2026a) rather than raw execution timesteps (Section 3.2). **(2)** A **two-stage joint-training strategy** uses random masks in the first stage and structured masks in the second stage to learn robust control over diverse and unseen control references (Section 3.4). **(3)** As a **training-free** alternative, we also adopt a **RePaint-style inpainting strategy** that can be applied during the inference-time denoising process of an off-the-shelf policy. We conduct experiments on 15 tasks across the ManiSkill, LIBERO-10, and real-world environments, covering six standard control references and randomly structured references. In sum, our method demonstrates several capabilities:

- **Versatile controllability.** Inpainting Policy supports diverse spatiotemporal references, including keypoints, flows, orientations, poses, and combinations of these reference types. In real-world experiments, it attains mean position and rotation errors of $44.3\,\mathrm{mm}$ and 7.8°, respectively. Meanwhile, our method improves the success rate on LIBERO-10 by 4.67 percentage points compared with the baseline. This proves that our policy enables arbitrary spatiotemporal controllability without compromising task completion.

- **Interpretability.** MAP (Masked Action Space) provides a human-readable and editable interface that bridges multi-dimensional spatiotemporal information from the physical world with robotic manipulation actions via the masked temporal Cartesian space. It explicitly indicates which timesteps and action dimensions are constrained.

- **Practicality.** Our method can be directly applied to diffusion- or flow matching-based policies in simulated or real-world environments. Built on a lightweight $49\mathrm{M}$-parameter DiT model, it achieves a 92.67% success rate and a control error of 0.023 (Table 1) in ManiSkill experiments. In real-world experiments, a $118\mathrm{M}$-parameter diffusion model achieves a success rate above 90%.

2 RELATED WORK

2.1 HETEROGENEOUS SPATIOTEMPORAL SIGNALS FOR ROBOT CONTROL

Robotic manipulation increasingly uses spatially grounded intermediate representations, including keypoints, trajectories, and keyposes, to guide action generation. Point-based methods provide sparse spatial guidance (Huang et al., 2024; Liu et al., 2024), while KALM uses task-relevant keypoints to define object-relative frames for action prediction (Fang et al., 2025). BiKC predicts keyposes to guide a low-level trajectory generator, and AnyGrasp provides grasp-pose proposals for downstream execution (Yu et al., 2024; Fang et al., 2023). Gemini Robotics-ER supports spatial reasoning and trajectory prediction, while RoboBrain and A0 produce affordance and trajectory predictions to guide manipulation (Team et al., 2025; Ji et al., 2025; Xu et al., 2026). Beyond sparse references, trajectory-based interfaces provide denser motion guidance for downstream action policies (Xu et al., 2024; Wen et al., 2024; Gu et al., 2023; Liu et al., 2025). Video-based models offer another route to control: V-JEPA 2-AC plans toward image goals through action-conditioned prediction in latent space, while UVA jointly learns video and action prediction (Assran et al., 2025; Li et al., 2025a).

Our work focuses on **unifying heterogeneous control references** as specified entries in a shared Cartesian action representation, allowing a single policy to reconstruct the remaining action entries across different reference types and their combinations.

2.2 INPAINTING FOR ROBOT CONTROL

Visual masking and inpainting support several aspects of robot learning. ROSIE augments robot observations through text-guided inpainting, while Mirage modifies robot appearance through cross-painting for embodiment transfer (Yu et al., 2023; Chen et al., 2024). Cloak masks the end-effector in wrist-camera observations without generative inpainting, whereas Masked Visual Actions uses partially revealed video trajectories for forward and inverse world modeling (Piseno et al., 2026; Alzayer et al., 2026). In motion generation, Kimodo supports pose-feature completion, Scheduled Inpainting balances motion preservation and editing, and MotionBricks performs latent in-betweening (Rempe et al., 2026; Agrawal et al., 2026; Wang et al., 2026b). In action space, RTC and DiscreteRTC preserve committed action prefixes while generating subsequent actions, enabling asynchronous execution through inpainting (Black et al., 2025; Wang et al., 2026a). HDP and DISCO use inpainting to impose trajectory-endpoint and semantic-keyframe constraints, respectively (Ma et al., 2024; Hao et al., 2025).

3 METHOD

Our method first introduces a unified controllable interface, the **MAP (Masked Action Space)**, for manipulation action generation, which converts heterogeneous spatiotemporal signals into specified action tokens and associated binary masks, thereby representing different control references in a common masked Cartesian action space (Section 3.1). Based on this representation, we develop an **Inpainting Policy** that reconstructs the masked action tokens and generates executable action chunks (Section 3.3). Our framework consists of three components. **(1)** A progress-guided sliding window aligns MAP with the current task progress and provides a temporally anchored MAP condition (Section 3.2). **(2)** Multi-modal joint training over heterogeneous mask families enables structured control and zero-shot generalization to unseen spatiotemporal compositions (Section 3.4). **(3)** We adopt RePaint (Lugmayr et al., 2022) as a training-free inference strategy, which repeatedly reinserts the specified action tokens during the denoising process of the diffusion-based policy, thereby improving adherence to the provided constraints (Section 3.5).

3.1 MASKED ACTION SPACE

We formulate reference-guided robotic manipulation as reconstructing complete actions from partial specifications in a temporal Cartesian action space.

We use normalized task progress $p' \in [0, 1]$ as the temporal coordinate, with reference alignment detailed in Section 3.2. Let $A^{\text{ref}} \in \mathbb{R}^{T_{\text{ref}} \times 7}$ denote the reference action-value array at discrete

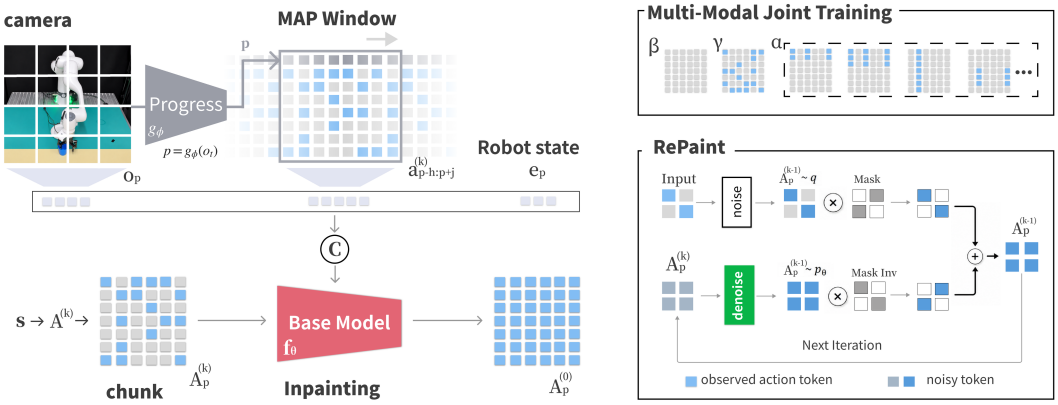


Figure 2: **Inpainting Policy**. Given camera observations and proprioceptive information, a progress network anchors a local MAP window and masked action chunk in the temporal Cartesian action space. The window is encoded into a feature vector to condition the base model, which predicts missing action tokens from the masked action chunk. Multi-modal joint training enables the policy to generalize to unseen spatiotemporal modalities.

progress locations p'_p, where $p = 0, \ldots, T_{\text{ref}} - 1$. At each location, the action vector is defined as $a_p = (x_p, y_p, z_p, \theta_{x,p}, \theta_{y,p}, \theta_{z,p}, d_p)$, where the first six dimensions describe the gripper pose and d_p denotes the gripper state. Each scalar action entry is a spatiotemporal action token. The spatiotemporal references are represented by a binary mask $M \in \{0,1\}^{T_{\text{ref}} \times 7}$ and the corresponding specified action tokens $M \odot A^{\text{ref}}$, where $M_{p,d} = 1$ indicates a supplied value and $M_{p,d} = 0$ indicates an unspecified entry.

Mapping Control References to Masks. The mapping $s \to A^{(k)}$ converts control references s into MAP constraints on the action chunk (Figure 2). As illustrated in Figure 1(C), different distributions of M instantiate different control signals. These references can be continuous or discrete along the temporal axis. Structured mask types include 2D/3D keypoints, 2D/3D flow, keyposes, orientations, gripper states, and mixed-modality conditions formed by randomly combining these references. Here, 2D references specify Cartesian x, y coordinates and flow references denote position trajectories; image-space references must first be mapped into the action coordinate system. For a reference at progress p', let $p^\star = \arg\min_p |p'_p - p'|$ denote its discrete reference index. For example, a 3D keypoint activates $M_{p^\star,\{1,2,3\}} = 1$, while an orientation reference activates $M_{p^\star,\{4,5,6\}} = 1$. MAP also supports unstructured mask types, as shown in Figure 1(B), such as specifying the gripper state at a particular manipulation stage, constraining the end-effector state along the z-axis, or specifying a rotation angle on a particular plane.

3.2 PROGRESS-GUIDED SLIDING WINDOW OF MAP

We use normalized task progress $p' \in [0,1]$ to align references with policy execution because execution speed can vary across rollouts, making raw timesteps unreliable for identifying corresponding task stages (see the related ablation study in Supplementary Section 1.5). Given the current observation $o_t = (I_t, e_t)$, where I_t denotes camera observations and e_t denotes the robot state, a progress network predicts $p'_t = g_\phi(o_t) \in [0,1]$. The corresponding reference index is $p_t = \arg\min_p |p'_p - p'_t|$.

The progress estimate selects a local reference window $\mathcal{W}_t = [a_{p_t-h}, \ldots, a_{p_t}, \ldots, a_{p_t+j}]$, where h and j specify the past and future context lengths. The encoder receives only the specified entries and their visibility masks; boundary handling and window-length conventions are detailed in Supplementary Section 3.1. The window associates reference constraints with the current task stage and provides nearby reference context for action generation. Its temporal coverage is controlled by the backward and forward window lengths, whose effect is evaluated in Supplementary Section 1.2.

3.3 INPAINTING POLICY

As illustrated in Figure 2, IP generates complete Cartesian action chunks conditioned on visual observations, robot state, and the local MAP window selected in Section 3.2. A 2D convolutional encoder maps the local MAP window to a 128-dimensional feature vector, which is concatenated with visual and proprioceptive features to condition the denoising network (see Supplementary Sections 3.6 and 1.1 for architecture details and encoder comparisons). Let $A^{(0)} \in \mathbb{R}^{H\times 7}$ denote a clean action chunk and $A^{(k)} = \sqrt{\bar{\alpha}_k}A^{(0)} + \sqrt{1-\bar{\alpha}_k}\epsilon$ its noisy version at diffusion step k, where $\bar{\alpha}_k$ is the cumulative signal-retention coefficient and $\epsilon \sim \mathcal{N}(0, I)$. The denoising network predicts $\widehat{\epsilon} = f_\theta(A^{(k)}, k, C_t)$, where C_t contains the observation features and the encoding of $\mathcal{W}_t$. Let $M_t^{\text{chunk}} \in \{0,1\}^{H\times 7}$ denote the reference visibility mask aligned with the target action chunk. The training objective is

$$\mathcal{L}_{\text{MAP}} = \mathbb{E}\left[w_m \left\|M_t^{\text{chunk}} \odot (\epsilon - \widehat{\epsilon})\right\|_F^2 + w_u \left\|(1 - M_t^{\text{chunk}}) \odot (\epsilon - \widehat{\epsilon})\right\|_F^2\right],$$

where w_m and w_u control the relative importance of known and unknown regions, and the expectation is over training examples, sampled masks, diffusion steps, and Gaussian noise. In practice, we use a weighted reconstruction objective to encourage consistency with specified references while learning to reconstruct unspecified actions.

3.4 MULTIMODAL JOINT TRAINING

To enable one model to handle diverse spatiotemporal conditions, we construct a multi-modal masked action dataset from demonstrations. As illustrated in Figure 2, we train the Inpainting Policy with three complementary mask patterns: α, β, and γ, each targeting a distinct aspect of masked action reconstruction.

The α-pattern consists of structured spatiotemporal control signals, including 2D keypoints/flow, 3D keypoints/flow, keyposes, orientation, and gripper-state conditions. These masks represent common controllable action-generation interfaces, where partial Cartesian action tokens are specified as subgoals or constraints. Joint training over α enables the policy to follow diverse structured conditions within a unified action space.

The β-pattern corresponds to the no-known-context case, where no action reference is provided. In this setting, the model reduces to plain action generation by the diffusion policy. Including β encourages robustness when test-time conditions are sparse, such as low-frequency control signals.

The γ-pattern introduces random token-level masks. Since structured masks are special cases of random masking, γ promotes token-level generalization across MAP, especially under OOD patterns. However, experiments show that random-mask training may underperform structured joint training when training data is limited.

We sample a mask family $c \sim \rho(c)$, where $c \in \{\alpha, \beta, \gamma\}$, and then draw a mask $M \sim p(M \mid c)$. The distribution $\rho(c)$ controls the relative frequency of structured, no-reference, and random-mask examples during policy training. Mask families and compositions are detailed in Supplementary Sections 3.4–3.5. For LIBERO-10, we first train with random masks and then continue with a mixture of structured masks, as detailed in Supplementary Sections 2.2 and 1.4.

We find that, after joint training, the model can follow compositions of different spatiotemporal constraints and generaloize to an OOD mask pattern, formed by combining several standard patterns but never observed during training (Figure 5).

3.5 INPAINTING VIA REPAINT

We adopt a RePaint-style inference strategy (Lugmayr et al., 2022) that repeatedly reinserts specified reference values during denoising without additional policy training. Given clean reference values $R_t \in \mathbb{R}^{H\times 7}$ aligned with the target action chunk, we sample their noisy version through forward diffusion. The binary mask M_t^{chunk} specifies the observed action tokens, while $1 - M_t^{\text{chunk}}$ indicates the unknown tokens to be reconstructed.

At each reverse denoising step k, we obtain a denoised proposal $\widetilde{A}^{(k-1)} = \mathcal{D}_\theta(A^{(k)}, k, C_t)$, noise the reference values to the corresponding level as $R_t^{(k-1)} = \sqrt{\bar{\alpha}_{k-1}}R_t + \sqrt{1-\bar{\alpha}_{k-1}}\xi_k$, and update

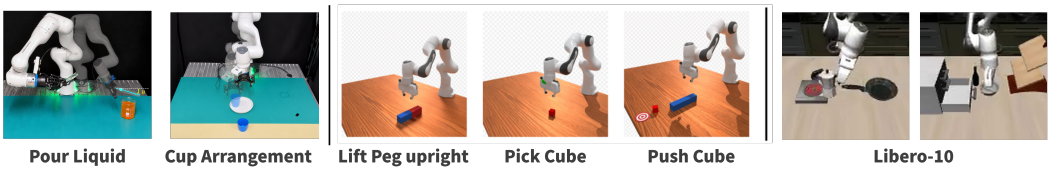


Figure 3: Simulation and real-world tasks.

the action chunk by replacing the known entries:

$$A^{(k-1)} = M_t^{\text{chunk}} \odot R_t^{(k-1)} + (1 - M_t^{\text{chunk}}) \odot \widetilde{A}^{(k-1)},$$

where $\mathcal{D}_\theta$ denotes one reverse diffusion step using the noise prediction $f_\theta(A^{(k)}, k, C_t)$, $\xi_k \sim \mathcal{N}(0, I)$, and C_t denotes the conditioning features, including visual features, robot state, and the MAP window.

The final action chunk is obtained by overwriting the known tokens with clean reference values at the last denoising step ($\bar{\alpha}_0 = 1$). This overwrite operation is applied at every denoising step, ensuring that the final sampled action chunk strictly preserves the specified reference values rather than merely conditioning on them.

4 EXPERIMENTS

Our experiments focus on three questions: (1) What information in physical references matters for action reconstruction? (2) Can the model generalize to unseen, out-of-distribution mask patterns? (3) How does each component contribute to the performance of our method? We evaluate our policy on 15 tasks across ManiSkill, LIBERO-10, and real-world environments, demonstrating diverse manipulation skills and long-horizon capabilities. Figure 3 provides an overview of the simulation and real-world tasks.

Dataset Construction. For the simulation experiments, we construct action references by masking demonstration trajectories, without collecting additional reference annotations. Pairing each demonstration with multiple mask configurations provides diverse reference conditions while preserving its observations and target actions. As shown in Figure 1(C), structured masks reveal modality-specific coordinate groups at selected reference locations for 2D/3D points, 7D poses, and orientations, or retain position coordinates along reference trajectories for 2D/3D flow. Random masks instead reveal individual scalar action entries across reference locations and action dimensions (Section 3.1). We additionally evaluate `Mix0`, a composition excluded from joint training that combines dense 2D flow, one 7D pose, and three 3D points (see Supplementary Section 2.4).

Progress Module. Our progress module adapts SARM (Chen et al., 2026a) to estimate task progress from frozen CLIP image and text features and proprioceptive states. The module is trained separately from the policy and uses an eight-layer Transformer with a hidden dimension of 768 and a temporal sampling interval of two frames.

Metrics. We report task success rate (SR) to evaluate task completion. In addition to SR, we introduce **Control Error (CE)** to quantify how closely the generated trajectory follows the prescribed control signals. Let $\tau = (a_1, a_2, \ldots, a_T)$ denote the generated action trajectory, where $a_t \in \mathbb{R}^7$, and let $\mathcal{C} = \{c_1, c_2, \ldots, c_K\}$ denote the control signals. For each signal c_ℓ, a selection matrix S_ℓ extracts the corresponding action dimensions. We define CE as the average distance from each control signal to its closest matching trajectory point: $\text{CE} = \frac{1}{K} \sum_{\ell=1}^{K} \min_{t \in \{1,\ldots,T\}} \|S_\ell \cdot a_t - c_\ell\|_2$. Lower CE indicates closer agreement with the prescribed control values.

Baseline. We compare our method with the following baselines. All methods are trained and evaluated using the same data, random seed, and initial states. **(1) Diffusion Policy (DP).** We use Diffusion Policy (Chi et al., 2025) with a U-Net backbone as the standard behavior cloning baseline,

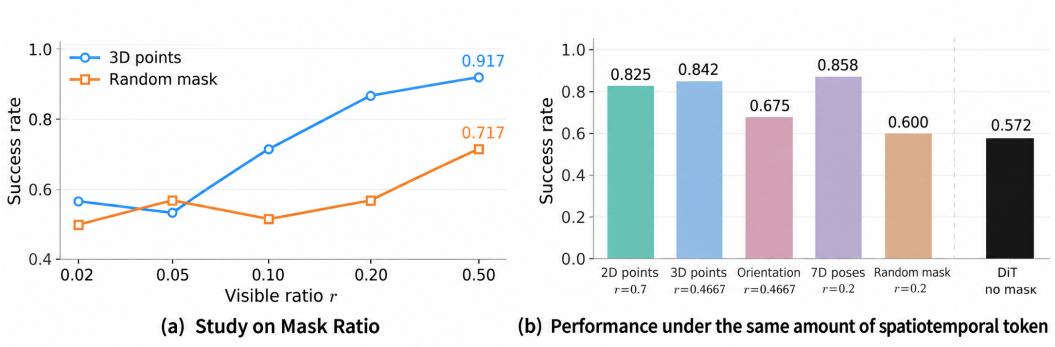


Figure 4: Effect of spatiotemporal information quantity and structure. (a) We vary the visible ratio to examine how the number of references affects task success. (b) We compare mask types with approximately equal numbers of spatiotemporal tokens, alongside a base policy (a DiT policy), to assess how constraint structure influences the performance of Inpainting Policy. Here, r denotes the visible ratio, i.e., the proportion of action information that is not masked in MAP.

generating actions without control references. **(2) DiT Policy.** We replace the U-Net with a Diffusion Transformer (DiT) (Peebles & Xie, 2023) to isolate the effect of the backbone. **(3) Spatiotemporal Conditioning (STC).** Following prior work on trajectory-, keypose-, and motion-conditioned action generation (Huang et al., 2025; Nasiriany et al., 2025; Wen et al., 2024; Gu et al., 2023; Fan et al., 2025a; Xu et al., 2024; Yu et al., 2024; Ma et al., 2024), STC flattens spatiotemporal signals and injects them into the diffusion transformer as conditioning features. It shares the backbone and training hyperparameters of Inpainting Policy to assess the necessity of the Inpainting Policy design.

4.1 STUDY ON ACTION INPAINTING

As shown in Figure 4, we investigate two aspects of action reconstruction in the temporal Cartesian space: **(1) Information quantity.** We examine how the amount of visible control information affects action reconstruction by comparing structured constraints, represented by 3D points, with random constraints. **(2) Constraint category.** We examine the effect of mask structure while approximately matching the number of spatiotemporal tokens across different reference types. We conduct this study on ManiSkill using 100 demonstrations per task and a DiT backbone with approximately 49M parameters. We vary the visible ratio and mask type and evaluate each configuration on 80 held-out episodes per task. Detailed training settings and mask budgets are provided in Supplementary Sections 2.2 and 2.4, respectively.

Mask Ratio and Mask-Type Analysis. Figure 4(a) shows that increasing the visible-action ratio generally improves task success, indicating that additional known action components provide stronger guidance for reconstructing executable behaviors. For the structured 3D-point mask, success increases from 57.0% at a ratio of 0.02 to 91.7% at 0.50. Although the random mask also improves with more visible actions, its performance is less stable and reaches only 71.7% at the highest ratio. This gap suggests that structured control signals convey more task-relevant information than randomly selected action components.

Figure 4(b) compares different mask types under approximately the same number of visible spatiotemporal tokens. Across all mask types, IP outperforms the unconditioned DiT policy. Positional and full-pose constraints provide the strongest guidance, whereas orientation-only constraints are less informative for task completion. The random-mask result further shows that semantically structured action references are more effective than an equivalent number of arbitrary visible tokens.

4.2 UNIFIED CONTROL VIA JOINT TRAINING

In this section, as shown in Table 1, we investigate whether a single policy can follow diverse control references through joint training on versatile spatiotemporal masks. We assess task completion

Table 1: **Unified control via joint training.** Each policy checkpoint is evaluated across reference types using the average top-three picks of success rate and control error (CE). ManiSkill uses a separate policy for each task, whereas LIBERO-10 uses a shared multitask policy.

	Multi-task (LIBERO-10)		Single-task (ManiSkill)				
Method	**Avg. SR (%) ↑**	**CE(all)↓**	**Lift Peg SR (%)↑**	**Pick Cube SR (%)↑**	**Poke Cube SR (%)↑**	**Avg. SR (%) ↑**	**CE(all)↓**
DP	60.00	—	86.33	69.33	69.67	75.11	—
DiT Policy	59.67	—	94.67	87.33	**93.00**	91.67	—
STC	50.67	0.060	72.67	65.67	91.67	76.67	0.041
IP	**64.67**	0.045	**100.00**	**90.33**	87.67	**92.67**	0.023
IP-RePaint	51.33	**0.026**	95.80	82.13	62.20	80.04	**0.014**

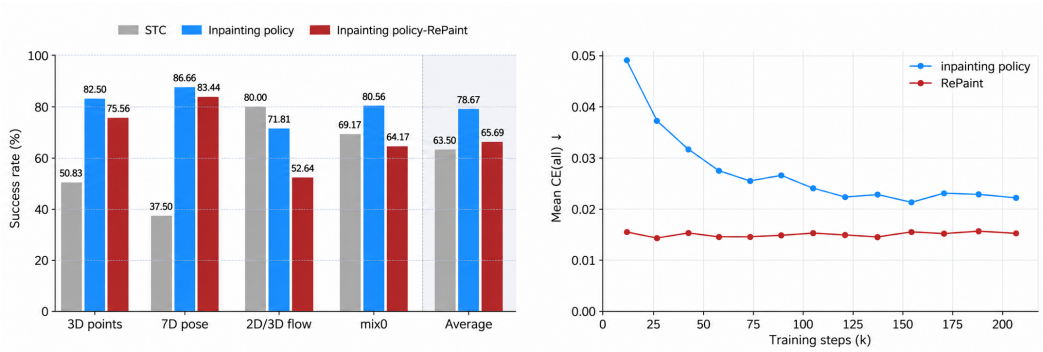


Figure 5: **Mask-type breakdown in joint training.** Left: We compare success rates across reference types, including the unseen mixed mask `Mix0`, to evaluate controllability and compositional generalization. Right: Control error decreases as training progresses.

and reference adherence through success rate and control error. We evaluate joint training of the Inpainting Policy across diverse constraint types in two settings: single-task policies with a DiT backbone on ManiSkill, and a shared language-conditioned multitask policy with a U-Net backbone on LIBERO-10. IP-RePaint is applied directly at inference time to the baseline DP and DiT policies for Versatile control.

Joint-Training. On LIBERO-10, we adopt **two-stage joint training**. First, we train a shared policy across all ten tasks with random masks; second, we continue training with a mixture of structured masks with different reference modalities and sparsity levels. At evaluation time, Mix0 serves as an unseen mask type. We apply **mask-wise data augmentation** by pairing each demonstration with multiple mask configurations, yielding diverse reference signals with a limited number of demonstrations. See Supplementary Sections 2.2 and 2.4 for the training settings and Mix0 construction, and Section 1.4 for the ablation study.

Joint-Training Performance. Table 1 shows that STC performs worse than its corresponding base policies, indicating that directly conditioning action generation on control signals can interfere with task SR. In contrast, Inpainting Policy improves task completion while achieving lower control error, performing well in both multi-task and single-task settings and achieving the highest average success rates of 64.67% and 92.67%, respectively. The training-free RePaint variant further reduces CE with relative reductions of 42.2% and 39.1%, respectively, demonstrating a trade-off between control accuracy and task completion.

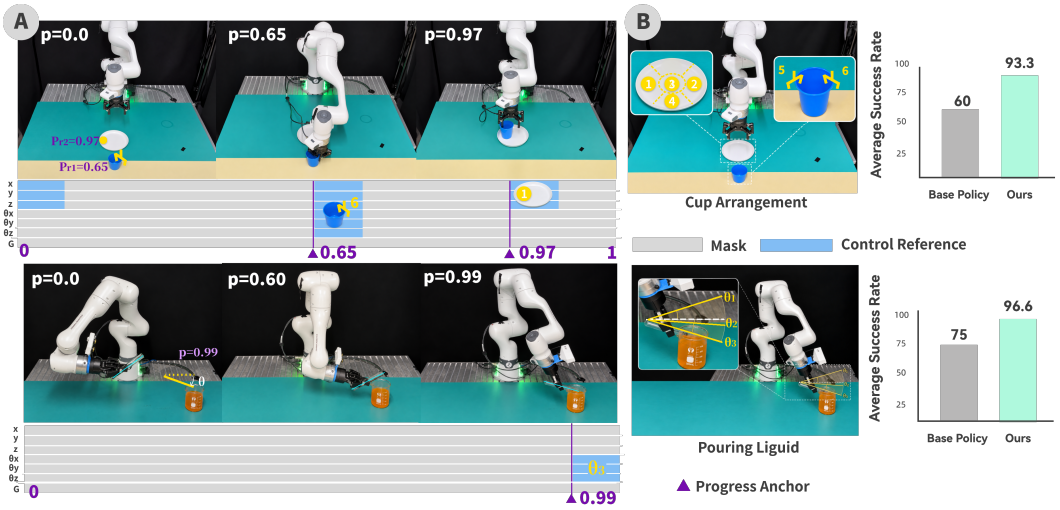


Figure 6: **Real-world spatiotemporal editing.** Here, p_r denotes the progress associated with a user-provided reference, whereas p denotes the robot's current execution progress. The rollouts and corresponding MAP masks illustrate how these references guide execution.

Mask-Type Breakdown and Training Dynamics. Figure 5 shows that Inpainting Policy provides the strongest average control performance across all control types, especially on the unseen `Mix0` mask (80.56%). STC performs strongly on dense 2D/3D flow (80.00%), but degrades sharply under sparse constraints. The right plot shows that Inpainting Policy progressively improves its control capability during training, with CE decreasing from approximately 0.049 to 0.022. In contrast, RePaint maintains the lowest CE, around 0.014–0.015, across all checkpoints. Notably, since the `Mix0` composition is absent from the training set, its high success rate indicates that the policy can **compose learned mask patterns** without additional finetuning.

4.3 CONTROL WITH REAL-WORLD REFERENCES

Real World Setting. For real-world experiments, we use Franka R3 robotic arms with two-finger grippers. One calibrated RealSense D455 camera provides the third-person view, and one ZED-M camera provides the wrist view. The robotic arm operates at a policy frequency of 15 Hz using absolute end-effector commands. We train task-specific Inpainting Policies (IP) and Diffusion Policy (DP) as the base policy. See Supplementary Section 2.6 for real-world implementation details.

Human-in-the-Loop Control. The **Robotic Spatiotemporal Editing** system accepts a variable number of user-provided references, each associated with a reference type and a progress value p' that specifies when the constraint should be applied. The system converts all references into a unified **Masked Action Space (MAP)** for policy inference.

Tasks. We evaluate whether IP can complete the tasks while successfully satisfying additional spatial and orientational objectives with references including keypoints, flows, poses, and orientations, under both sparse and dense configurations. As shown in Figure 6 (B), in the cup-arrangement task, these references determine how the cup is grasped and where it is placed on the plate. In the liquid-pouring task, orientation references specify the tube's tilt angle and thereby regulate the dispensed volume.

Result. As shown in Figure 6, with three or more reference points, the policy achieves success rates above 93%, while the cup-to-reference-point distance ranges from 18.2 to 63.7 mm, with an average of 44.3 mm. In the liquid-pouring task, the tube-tilt error remains below 8°, with a mean of 7.8°. These results show that providing explicit control constraints substantially improves task completion and reference adherence: denser references produce smoother motion and smaller deviations

from the target. In contrast, when only a single reference is provided at the final progress point, the task completion rate falls below 30%, revealing a limitation of sparse late-stage supervision and an important direction for improving the subsequent editing system. Supplementary Section 2.6 provides additional recorded SR and normalized CE results (Tables 8–10), plus deployment tips.

5 CONCLUSION AND FUTURE WORK

Conclusion. We presented a unified framework for spatiotemporal action editing that augments policies with DiT or U-Net backbones with a unified control interface over heterogeneous spatiotemporal references. Without requiring additional control-specific data or increasing the backbone training budget, our method preserves the original task completion ability while enabling users to edit and compose spatiotemporal action constraints through masked action inpainting. Our inpainting formulation is model-agnostic and may be compatible with a broad range of foundation action models, including WAM and flow-matching-based VLAs, which we leave for future empirical study.

Limitations. Our current inpainting implementation may not be optimal; alternatives such as CFG and ControlNet-style conditioning remain unexplored. Future work will study large-scale BERT-style random-mask pretraining for foundation action models and evaluate zero-shot editing under more unstructured control specifications.

AI USE DISCLOSURE

In this work, we used generative AI tools to assist with translating and polishing the manuscript, drafting and editing parts of the paper, improving the readability of the presentation, modifying scientific figures, interpreting experimental results, and writing and modifying code.

We did not use generative AI tools to generate synthetic datasets, develop theoretical models or conceptual frameworks, formulate mathematical claims, provide critical ingredients for proving mathematical claims, write proofs, propose or refine hypotheses, design or provide feedback on the research methodology or experiments, clean or reformat datasets, or perform qualitative or thematic data analysis.

All AI-assisted content was reviewed and verified by the authors. The authors take full responsibility for the final content of this submission, including all text, claims, experimental results, code, and figures.

ETHICS STATEMENT

This work studies controllable robot manipulation using public simulation benchmarks and controlled real-world robotic experiments. It does not involve human subjects, personal data, sensitive information, or high-risk applications. The real-world experiments were conducted in a controlled laboratory environment following standard robot-safety procedures. We have read and followed the ICLR Code of Ethics and take responsibility for the ethical conduct of this research.

REPRODUCIBILITY STATEMENT

To support reproducibility, the supplementary material describes the masked-action formulation and inference procedure in *Additional Method Details*, and the policy architectures, training settings, mask construction, evaluation metrics, and real-world setup in *Additional Implementation Details*. The *Ablation Studies* section reports the settings and evaluation protocols for the encoder, window-length, progress-module, training-schedule, and progress-alignment comparisons. These descriptions distinguish comparisons with matched settings from historical comparisons with differing training or evaluation conditions, and identify results aggregated across reference configurations and checkpoints.

REFERENCES

Dhruv Agrawal, Dominik Borer, Luca Vögeli, Robert Sumner, Martin Guay, and Jakob Buhmann. Interactive generative motion editing via scheduled inpainting. *arXiv preprint arXiv:2607.29133*, 2026.

Hadi Alzayer, Wenlong Huang, Haonan Chen, Christopher Luey, Lvmin Zhang, Maneesh Agrawala, Gordon Wetzstein, Li Fei-Fei, Yilun Du, Jiajun Wu, et al. Masked visual actions for unified world modeling. *arXiv preprint arXiv:2607.19343*, 2026.

Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

Kevin Black, Manuel Y Galliker, and Sergey Levine. Real-time execution of action chunking flow policies, 2025. *URL https://arxiv. org/abs/2506.07339*, 2025.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control, 2026. URL https://arxiv.org/abs/2410.24164.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin Stone, Clayton Tan, Huong Tran, Vincent Vanhoucke, Steve Vega, Quan Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-1: Robotics transformer for real-world control at scale, 2023. URL https://arxiv.org/abs/2212.06817.

Lawrence Yunliang Chen, Kush Hari, Karthik Dharmarajan, Chenfeng Xu, Quan Vuong, and Ken Goldberg. Mirage: Cross-embodiment zero-shot policy transfer with cross-painting. *arXiv preprint arXiv:2402.19249*, 2024.

Qianzhong Chen, Justin Yu, Mac Schwager, Pieter Abbeel, Yide Shentu, and Philipp Wu. Sarm: Stage-aware reward modeling for long horizon robot manipulation, 2026a. URL https://arxiv.org/abs/2509.25358.

Ruoqu Chen, Feixiang Ruan, Liu Cao, Zihao Wang, Botian Xu, Shiqin Tong, Jiajun Liu, Mingzhi Pei, Chenyu Zhang, Wanli Xing, Kaifeng Zhang, and Mengdi Xu. Dex-x: Learning visual-tactile dexterous manipulation from human videos with simulated interaction, 2026b. URL https://arxiv.org/abs/2609.07747.

Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

Shichao Fan, Quantao Yang, Yajie Liu, Kun Wu, Zhengping Che, Qingjie Liu, and Min Wan. Diffusion trajectory-guided policy for long-horizon robot manipulation. *IEEE Robotics and Automation Letters*, 2025a.

Yiguo Fan, Pengxiang Ding, Shuanghao Bai, Xinyang Tong, Yuyang Zhu, Hongchao Lu, Fengqi Dai, Wei Zhao, Yang Liu, Siteng Huang, Zhaoxin Fan, Badong Chen, and Donglin Wang. Long-vla: Unleashing long-horizon capability of vision language action model for robot manipulation, 2025b. URL https://arxiv.org/abs/2508.19958.

Hao-Shu Fang, Chenxi Wang, Hongjie Fang, Minghao Gou, Jirong Liu, Hengxu Yan, Wenhai Liu, Yichen Xie, and Cewu Lu. Anygrasp: Robust and efficient grasp perception in spatial and temporal domains. *IEEE Transactions on Robotics*, 39(5):3929–3945, 2023.

Xiaolin Fang, Bo-Ruei Huang, Jiayuan Mao, Jasmine Shone, Joshua B Tenenbaum, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Kalm: Keypoint abstraction using large models for object-relative imitation learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8307–8314. IEEE, 2025.

Shenyuan Gao, William Liang, Kaiyuan Zheng, Ayaan Malik, Seonghyeon Ye, Sihyun Yu, Wei-Cheng Tseng, Yuzhu Dong, Kaichun Mo, Chen-Hsuan Lin, Qianli Ma, Seungjun Nah, Loic Magne, Jiannan Xiang, Yuqi Xie, Ruijie Zheng, Dantong Niu, You Liang Tan, K. R. Zentner, George Kurian, Suneel Indupuru, Pooya Jannaty, Jinwei Gu, Jun Zhang, Jitendra Malik, Pieter Abbeel, Ming-Yu Liu, Yuke Zhu, Joel Jang, and Linxi "Jim" Fan. Dreamdojo: A generalist robot world model from large-scale human videos, 2026. URL https://arxiv.org/abs/2602.06949.

Jiayuan Gu, Sean Kirmani, Paul Wohlhart, Yao Lu, Montserrat Gonzalez Arenas, Kanishka Rao, Wenhao Yu, Chuyuan Fu, Keerthana Gopalakrishnan, Zhuo Xu, et al. Rt-trajectory: Robotic task generalization via hindsight trajectory sketches. *arXiv preprint arXiv:2311.01977*, 2023.

Ce Hao, Kelvin Lin, Zhiwei Xue, Siyuan Luo, and Harold Soh. Disco: Language-guided manipulation with diffusion policies and constrained inpainting. *IEEE Robotics and Automation Letters*, 2025.

Kefeng Huang, Tingguang Li, Yuzhen Liu, Zhe Zhang, Jiankun Wang, and Lei Han. Vlm-tdp: Vlm-guided trajectory-conditioned diffusion policy for robust long-horizon manipulation. *arXiv preprint arXiv:2507.04524*, 2025.

Wenlong Huang, Chen Wang, Yunzhu Li, Ruohan Zhang, and Li Fei-Fei. Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. *arXiv preprint arXiv:2409.01652*, 2024.

Joel Jang, Seonghyeon Ye, Zongyu Lin, Jiannan Xiang, Johan Bjorck, Yu Fang, Fengyuan Hu, Spencer Huang, Kaushil Kundalia, Yen-Chen Lin, Loic Magne, Ajay Mandlekar, Avnish Narayan, You Liang Tan, Guanzhi Wang, Jing Wang, Qi Wang, Yinzhen Xu, Xiaohui Zeng, Kaiyuan Zheng, Ruijie Zheng, Ming-Yu Liu, Luke Zettlemoyer, Dieter Fox, Jan Kautz, Scott Reed, Yuke Zhu, and Linxi Fan. Dreamgen: Unlocking generalization in robot learning through video world models, 2025. URL https://arxiv.org/abs/2505.12705.

Yuheng Ji, Huajie Tan, Jiayu Shi, Xiaoshuai Hao, Yuan Zhang, Hengyuan Zhang, Pengwei Wang, Mengdi Zhao, Yao Mu, Pengju An, et al. Robobrain: A unified brain model for robotic manipulation from abstract to concrete. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1724–1734. IEEE, 2025.

Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model, 2025a. URL https://arxiv.org/abs/2503.00200.

Yanbang Li, Ziyang Gong, Haoyang Li, Xiaoqi Huang, Haolan Kang, Guangping Bai, and Xianzheng Ma. Robotic visual instruction, 2025b. URL https://arxiv.org/abs/2505.00693.

Fangchen Liu, Kuan Fang, Pieter Abbeel, and Sergey Levine. Moka: Open-world robotic manipulation through mark-based visual prompting. *arXiv preprint arXiv:2403.03174*, 2024.

Mingyu Liu, Zheng Huang, Xiaoyi Lin, Muzhi Zhu, Canyu Zhao, Yating Wang, Haoyi Zhu, Hao Chen, and Chunhua Shen. Gae: Unleashing physical potential of vlm with generalizable action expert. In *Forty-third International Conference on Machine Learning*, 2025.

Andreas Lugmayr, Martin Danelljan, Andres Romero, Fisher Yu, Radu Timofte, and Luc Van Gool. Repaint: Inpainting using denoising diffusion probabilistic models, 2022. URL https://arxiv.org/abs/2201.09865.

Xiao Ma, Sumit Patidar, Iain Haughton, and Stephen James. Hierarchical diffusion policy for kinematics-aware multi-task robotic manipulation. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 18081–18090. IEEE, 2024.

Adithyavairavan Murali, Balakumar Sundaralingam, Yu-Wei Chao, Wentao Yuan, Jun Yamada, Mark Carlson, Fabio Ramos, Stan Birchfield, Dieter Fox, and Clemens Eppner. Graspgen: A diffusion-based framework for 6-dof grasping with on-generator training, 2025. URL https://arxiv.org/abs/2507.13097.

Soroush Nasiriany, Sean Kirmani, Tianli Ding, Laura Smith, Yuke Zhu, Danny Driess, Dorsa Sadigh, and Ted Xiao. Rt-affordance: Affordances are versatile intermediate representations for robot manipulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8249–8257. IEEE, 2025.

William Peebles and Saining Xie. Scalable diffusion models with transformers, 2023. URL https://arxiv.org/abs/2212.09748.

Michael Piseno, Guy Tevet, and C Karen Liu. Cloak: Zero-shot cross-embodiment manipulation by masking the end-effector from the vla. *arXiv preprint arXiv:2606.22836*, 2026.

Davis Rempe, Mathis Petrovich, Ye Yuan, Haotian Zhang, Xue Bin Peng, Yifeng Jiang, Tingwu Wang, Umar Iqbal, David Minor, Michael de Ruyter, et al. Kimodo: Scaling controllable human motion generation. *arXiv preprint arXiv:2603.15546*, 2026.

Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Travis Armstrong, Ashwin Balakrishna, Robert Baruch, Maria Bauza, Michiel Blokzijl, et al. Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.

Pengcheng Wang, Kaiwen Hong, Chensheng Peng, Katherine Driggs-Campbell, Masayoshi Tomizuka, Chenfeng Xu, and Chen Tang. Discretertc: Discrete diffusion policies are natural asynchronous executors. *arXiv preprint arXiv:2604.25050*, 2026a.

Tingwu Wang, Olivier Dionne, Michael De Ruyter, David Minor, Davis Rempe, Kaifeng Zhao, Mathis Petrovich, Ye Yuan, Chenran Li, Zhengyi Luo, et al. Motionbricks: Scalable real-time motions with modular latent generative model and smart primitives. *ACM Transactions on Graphics (TOG)*, 45(4):1–22, 2026b.

Chuan Wen, Xingyu Lin, John So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning, 2024. URL https://arxiv.org/abs/2401.00025.

Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface, 2024. URL https://arxiv.org/abs/2407.15208.

Rongtao Xu, Jian Zhang, Minghao Guo, Youpeng Wen, Haoting Yang, Min Lin, Jianzheng Huang, Zhe Li, Kaidong Zhang, Liqiong Wang, Yuxuan Kuang, Meng Cao, Feng Zheng, and Xiaodan Liang. A0: An affordance-aware hierarchical model for general robotic manipulation, 2026. URL https://arxiv.org/abs/2504.12636.

Dongjie Yu, Hang Xu, Yizhou Chen, Yi Ren, and Jia Pan. Bikc: Keypose-conditioned consistency policy for bimanual robotic manipulation, 2024. URL https://arxiv.org/abs/2406.10093.

Tianhe Yu, Ted Xiao, Austin Stone, Jonathan Tompson, Anthony Brohan, Su Wang, Jaspiar Singh, Clayton Tan, Dee M, Jodilyn Peralta, Brian Ichter, Karol Hausman, and Fei Xia. Scaling robot learning with semantically imagined experience, 2023. URL https://arxiv.org/abs/2302.11550.

Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models, 2023. URL https://arxiv.org/abs/2302.05543.

Supplementary Material

1 Ablation Studies

We study how the MAP encoder, forward window length, progress module, training schedule, and progress source affect Inpainting Policy (IP). These comparisons inform the design choices in the main paper. Some settings differ in more than one factor, so their results cannot isolate each factor's contribution; we describe these differences alongside each experiment. Top-k SR is the average of the k highest evaluated SR values within the same run. CE(all), CE(success), and CE(failed) denote control error over all, successful, and failed rollouts, respectively; lower CE is better.

1.1 MAP Window Encoding and Feature Dimension

On ManiSkill PickCube, we compare a 1DConv encoder with a 32-dimensional output against 2DConv encoders with output dimensions of 32, 64, and 128. Each output forms the MAP conditioning vector. All runs use 100 demonstrations, seed 1, observation/execution/prediction horizons of 2/8/16, a forward window of 32, and the `points@1` mask configuration. The IP Transformer uses a hidden dimension of 512, six encoder and six decoder blocks, and a feed-forward dimension of 2048.

Table 2: **MAP encoder and feature-dimension comparison on PickCube.** Training budgets, batch sizes, and evaluation counts are not fully matched. Bold marks the highest recorded Top-3 SR.

Encoder	Feature dimension	Top-3 SR (%) ↑	Last evaluation step
1DConv	32	12.00	155k
2DConv	32	19.00	180k
2DConv	64	14.67	175k
2DConv	128	**22.00**	275k

At output dimension 32, 2DConv reaches a Top-3 SR of 19.00%, compared with 12.00% for 1DConv (Table 2). The 2DConv results are 19.00%, 14.67%, and 22.00% at dimensions 32, 64, and 128, respectively. The 128-dimensional encoder used in the main paper has the highest recorded SR, but it is also evaluated later, at 275k updates. The results therefore reflect differences in both the encoder and its training configuration.

1.2 MAP Window Length

We test forward MAP windows of 52, 64, and 70 reference locations on ManiSkill PickCube-v1, keeping the training and evaluation settings fixed. Each model uses the same 100 demonstrations, 200k training updates, a batch size of 128, observation/execution/prediction horizons of 2/8/16, and mean-squared-error loss with mean reduction. The backward window length is zero, and the forward window includes the current reference anchor; its length is separate from the IP prediction horizon. All three models use a 128-dimensional 2DConv encoder, the `points@1` mask configuration, and the same STPM checkpoint, `reward_best.pt`.

Table 3: **MAP window-length comparison on PickCube-v1.** SR statistics summarize three training seeds; end SR measures final-step success. CE(all) is the aggregate error at 200k. Bold marks the highest mean success rates and lowest CE.

Forward length	Top-3 SR (%) ↑	200k SR (%) ↑	200k end SR (%) ↑	200k CE(all) ↓
52	**70.22 ± 4.25**	**63.33 ± 3.79**	**36.33 ± 3.06**	**0.02317**
64	61.33 ± 3.00	55.67 ± 5.03	33.33 ± 4.93	0.03281
70	60.22 ± 3.40	55.33 ± 2.89	32.33 ± 4.73	0.03198

The 52-location window performs best across the reported metrics (Table 3). Its Top-3 SR is 70.22%, exceeding the 64- and 70-location windows by 8.89 and 10.00 percentage points, respectively. At the final checkpoint, it also has the highest SR (63.33%) and end SR (36.33%), along with the lowest CE(all) (0.02317). Both the Top-3 and final-checkpoint results favor a forward length of 52 among the settings tested on PickCube-v1. Even with the same mask rule, however, window length can affect both temporal coverage and how much reference information is visible.

1.3 PROGRESS-MODULE CONFIGURATION

To compare three STPM configurations on LIBERO-10, we keep the IP checkpoint fixed at 190k and replace only STPM, without retraining IP. We evaluate each configuration on 50 episodes per seed, using evaluation seeds 1000, 2000, and 3000. Table 4 lists each module's architecture, observation history, frame gap, and checkpoint-selection criterion.

Table 4: **STPM configuration comparison on LIBERO-10 with a fixed 190k IP checkpoint.** SR is mean $\pm$ population standard deviation over three evaluation seeds. Architecture, observation history, and checkpoint selection vary together. Bold marks the highest mean SR.

STPM	Dimension / layers / heads	Obs. / gap	Checkpoint selection	SR (%) $\uparrow$
v3	512 / 4 / 8	1 / 1	Sequence best	**74.00 $\pm$ 0.00**
v5	768 / 8 / 12	6 / 2	Endpoint best	70.67 $\pm$ 5.73
v6	544 / 5 / 8	6 / 2	Endpoint best	69.33 $\pm$ 3.40

STPM v3 gives the highest mean SR at 74.00%, followed by v5 at 70.67% and v6 at 69.33%. All three modules support task completion, with mean SRs between 69.33% and 74.00%. The larger models offer no improvement for this IP checkpoint, although the comparison also changes observation history and checkpoint selection along with architecture.

1.4 JOINT AND TWO-STAGE TRAINING

We compare joint and two-stage training on LIBERO-10 using 500 source demonstrations and 50 evaluation trajectories. Both runs use STPM v4, a forward window of 64, a 128-dimensional MAP encoding, a U-Net IP backbone, and seed 1000, with the short-window option disabled. Joint training uses a four-slot reference mixture throughout steps 0–180k. Two-stage training uses random masks for steps 0–90k, then switches to the same four-slot mixture for steps 90k–180k. We also report an independent two-stage run with 1,000 source demonstrations (100 per task), 50 evaluation episodes, a global batch size of 96, and STPM v4 in Table 5. This run uses the same short-window setting, forward window length, and MAP feature dimension.

Table 5: **Training-schedule comparison on LIBERO-10.** All SR values are percentages; Top-1 step identifies the best evaluated checkpoint. Bold marks the higher SR within the 500-demonstration comparison. The 1,000-demonstration run is an independent experiment.

Source demos	Training	Top-1 SR $\uparrow$	Top-1 step	Top-3 SR $\uparrow$	Top-5 SR $\uparrow$
500	Joint	56.00	180k	50.00	47.60
500	Two-stage	**62.00**	105k	**56.00**	**54.40**
1,000	Two-stage	100.00	150k	97.33	96.00

With 500 demonstrations, two-stage training reaches Top-3 and Top-5 SRs of 56.00% and 54.40%, compared with 50.00% and 47.60% for joint training. The improvement supports the schedule used in the main paper: random-mask pretraining followed by structured joint training. The separate run with 1,000 demonstrations reaches its best evaluated SR of 100.00% at 150k, with Top-3 and Top-5 SRs of 97.33% and 96.00%. This result suggests a benefit from more demonstrations, though dataset size is not the only factor that differs between the independent runs.

1.5 Estimated Progress versus Rollout Timestep

We compare STPM-based alignment with a rollout-timestep baseline on PickCube, using the same IP checkpoint, 100 evaluation trajectories, seed 1, and a maximum episode length of 160. For learned alignment, we use the STPM checkpoint `STPM_pickcube/reward_best.pt`. The baseline selects the MAP window using $p'_t = t/(160-1)$, where t is the zero-based rollout step, in place of the estimated progress. Both modes retain the progress column in the reference trajectory.

Table 6: **Progress-source comparison with a fixed PickCube IP checkpoint and 100 evaluation trajectories.** SR is reported as a percentage. Bold marks the higher success rate or lower CE in each column. Successful and failed subsets can differ between modes.

Progress source	SR ↑	CE(all) ↓	CE(success) ↓	CE(failed) ↓
STPM	**83.00**	**0.02912**	**0.02675**	**0.04073**
Rollout timestep	67.00	0.03983	0.03297	0.05377

STPM alignment raises success-once SR from 67% to 83%, a gain of 16 percentage points, while reducing CE(all) from 0.03983 to 0.02912 (Table 6). CE is also lower for both successful and failed rollouts, consistent with closer reference adherence, though the rollouts in each subset can differ between modes. For this checkpoint and evaluation set, learned progress alignment improves both task success and overall reference accuracy.

2 Additional Implementation Details

2.1 Benchmarks, Data, and Baselines

Benchmarks and dataset construction. ManiSkill covers picking and placing objects, controlling object rotation, and pushing objects to target locations. LIBERO-10 comprises ten long-horizon manipulation tasks with visual observations and language instructions. Figure 3 (Simulation and real-world tasks) in the main paper summarizes the simulation and real-world tasks. We construct training and test examples by masking demonstrations, without collecting additional control references. Mask-wise data augmentation pairs each demonstration with multiple mask configurations while preserving its observations and target actions.

DP and DiT Policy. DP uses a U-Net backbone for observation-to-action generation without an explicit spatiotemporal control interface (Chi et al., 2025). DiT Policy replaces the U-Net with a Diffusion Transformer (Peebles & Xie, 2023), keeping the action prediction objective and training protocol unchanged to examine the effect of the backbone.

Spatiotemporal Conditioning (STC). Following methods that condition action generation on trajectories, keyposes, or motion guidance (Huang et al., 2025; Nasiriany et al., 2025; Wen et al., 2024; Gu et al., 2023; Fan et al., 2025a; Xu et al., 2024; Yu et al., 2024; Ma et al., 2024), STC flattens spatiotemporal control signals and injects them into a Diffusion Transformer as conditioning features. It shares the backbone and training hyperparameters of Inpainting Policy, allowing a comparison between direct reference conditioning and masked-action reconstruction. Within each comparison, methods use the same data, random seed, and initial states. We additionally evaluate IP-RePaint as a training-free inference variant of Inpainting Policy.

2.2 Policy Architectures and Training

Mask ratio. We investigate how the amount of visible control information affects action reconstruction by comparing structured constraints, represented by 3D points, with random constraints. We quantify the information provided by either constraint type using the spatiotemporal information budget N_{ST}, the number of visible scalar action entries. For a reference sequence of length T with action dimension $D = 7$, 3D-point and random constraints provide approximately $3rT$ and rTD visible entries, respectively. Here, the structured-mask ratio is defined over reference locations, whereas the random-mask ratio is defined over scalar action entries, following the mask settings

shown in the main paper. Equal visible ratios therefore do not imply equal information budgets; Section 2.4 gives the precise definitions and budget-matched settings.

Mask category. We next examine the effect of constraint structure while approximately matching the number of spatiotemporal tokens across constraint types. Each token corresponds to one visible action coordinate at one reference location. Thus, a 2D point contains two tokens, a 3D point contains three, and a 7D pose contains seven; complete 2D and 3D trajectories contain $2T$ and $3T$ tokens, respectively. Random constraints contribute one token per revealed action coordinate. We set $N_{\mathrm{ST}} = 0.2TD = 1.4T$ and use visible ratios of 0.70, $7/15$, 0.20, and 0.20 for 2D points, 3D points, 7D poses, and random masks, respectively. An orientation reference specifies three rotational coordinates and therefore uses the same visible ratio as a 3D point, $r_{\mathrm{orientation}} = 7/15$. For random constraints, the ratio is instead computed over all action entries: $r_{\mathrm{random}} = N_{\mathrm{ST}}/(TD)$.

Action-inpainting study. The mask study uses a DiT with six encoder blocks, six decoder blocks, a hidden dimension of 512, and approximately 49M trainable parameters. Each policy is trained on 100 demonstrations per task for 100,000 iterations, with a batch size of 64 and a learning rate of 10^{-4}. Each visibility and mask-type configuration is evaluated on 80 held-out episodes per task.

Single-task joint training on ManiSkill. Each task uses a separate policy trained on 500 demonstrations with heterogeneous spatiotemporal masks. The DiT architecture is the same as in the mask study. Training uses 300,000 updates, a global batch size of 128, and a learning rate of 5×10^{-5}.

Multi-task joint training on LIBERO-10. We train a shared language-conditioned policy on the official LIBERO-10 dataset. The policy uses a U-Net-based Diffusion Policy with channel dimensions of 512, 1024, and 2048, and a lightweight Transformer language encoder. Two-stage joint training first uses random masks across all ten tasks and then continues with a mixture of structured masks spanning different guidance modalities and sparsity levels. Training uses 180,000 updates, a global batch size of 96, and a learning rate of 10^{-4}. The fixed test set contains 100 trajectories derived from the official LIBERO-10 pruned states; masking these trajectories produces the evaluation references.

Progress module. We adapt SARM (Chen et al., 2026a) to estimate task progress from frozen CLIP image and text features and proprioceptive states. The module is trained separately and uses an eight-layer Transformer with a hidden dimension of 768 and a temporal sampling interval of two frames. We refer to the progress module as STPM in the configuration descriptions and comparisons below.

2.3 Control-Error Definitions

Simulation. Let $\tau = (a_1, \ldots, a_T)$ be a generated action trajectory, with $a_t \in \mathbb{R}^7$, and let $\mathcal{C} = \{c_1, \ldots, c_K\}$ contain the prescribed control signals. A selection matrix S_ℓ extracts the action dimensions specified by c_ℓ. We define

$$\mathrm{CE} = \frac{1}{K} \sum_{\ell=1}^{K} \min_{t \in \{1,\ldots,T\}} \left\| S_\ell a_t - c_\ell \right\|_2 . \tag{1}$$

Lower CE indicates closer agreement with the reference values. Because each signal is matched to its closest trajectory point, this metric measures reference proximity rather than alignment to a prescribed timestep. CE is not reported for the unconditioned DP and DiT baselines.

Real-world evaluation. For the real-world results in Section 4.3 of the main paper, we report task success rate, cup-to-reference-point distance in millimeters, and tube-tilt error in degrees. The distance and angle errors quantify task-specific physical outcomes; they are not interchangeable with normalized action-space CE. We report normalized CE separately from these physical errors. The evaluated tasks, reference configurations, and reported result ranges are described in Section 2.6.

Table 7: **Visible ratios for the budget-matched mask comparison, with $N_{\mathrm{ST}} = 1.4T$.** Structured-mask ratios are defined over reference locations; the random-mask ratio is defined over scalar action entries.

Reference type	Coordinates per reference location	Visible ratio
2D point	2	0.70
3D point	3	$7/15$
Orientation	3	$7/15$
7D pose	7	0.20
Random	—	0.20

2.4 Mask Construction and Information Budgets

Information budget. For a binary visibility mask $\mathbf{M} \in \{0,1\}^{T\times D}$, where T denotes the reference sequence length, $D = 7$, and $M_{p,d} = 1$ indicates a visible coordinate, the spatiotemporal information budget is

$$N_{\mathrm{ST}} = \|\mathbf{M}\|_0 = \sum_{p=0}^{T-1}\sum_{d=1}^{D} M_{p,d}. \tag{2}$$

A spatiotemporal token denotes one visible action coordinate at one reference location. Thus, 2D points, 3D points, and 7D poses contain two, three, and seven tokens per selected reference location. Complete 2D and 3D trajectories contain $2T$ and $3T$ tokens, respectively. Random masks reveal individual action entries.

Visibility-ratio study. For 3D-point constraints, a visible ratio r reveals approximately $3rT$ scalar entries. For random masks, the same ratio reveals approximately rTD entries. Equal visible ratios therefore do not imply equal information budgets.

Budget-matched mask comparison. We set $N_{\mathrm{ST}} = 0.2TD = 1.4T$. For structured references revealing d coordinates per selected reference location, the corresponding visible ratio is

$$r_d = \frac{N_{\mathrm{ST}}}{dT}. \tag{3}$$

For random constraints, the ratio is computed over all action entries: $r_{\mathrm{random}} = N_{\mathrm{ST}}/(TD)$. Table 7 lists the resulting ratios. Budgets are matched approximately because the numbers of selected reference locations and entries are discrete.

Unseen mixed mask: `Mix0`. This pattern is excluded from joint training. It preserves (x, y) over the entire trajectory. At four randomly selected distinct reference locations, it additionally retains one full 7-dimensional action at one location and an (x, y, z) position at each of the other three locations. The resulting mask combines dense 2D flow, one 7D pose, and three 3D points, as described in the main paper. Evaluation uses the trained policy without fine-tuning.

2.5 Additional Joint-Training Results

Per-task performance. Table 8 reports per-task results on ManiSkill: Inpainting Policy achieves the highest SR on LiftPegUpright and PickCube, while DiT Policy performs best on PokeCube.

Mask-type breakdown. The evaluation in Figure 5 (Mask-type breakdown in joint training) in the main paper reports an average SR of 78.67% for Inpainting Policy and an SR of 80.56% on the unseen `Mix0` mask. IP-RePaint achieves 52.64% on dense 2D/3D flow and 64.17% on `Mix0`, illustrating the trade-off between reference adherence and task completion described in the main paper.

Table 8: **Per-task ManiSkill success rates (%).** Each task uses a separate policy trained with 500 demonstrations.

Method	LiftPeg Upright	Pick Cube	Poke Cube
DP	86.33	69.33	69.67
DiT Policy	94.67	87.33	**93.00**
STC	72.67	65.67	91.67
IP	**100.00**	**90.33**	87.67
IP-RePaint	95.80	82.13	62.20

Training dynamics. In Figure 5 (Mask-type breakdown in joint training) in the main paper, Inpainting Policy's CE decreases from approximately 0.049 to 0.022 during training. IP-RePaint maintains the lowest CE, approximately 0.014–0.015, across the evaluated checkpoints. In the aggregate results in Table 1 (Unified control via joint training) in the main paper, IP-RePaint reduces CE relative to Inpainting Policy by 42.2% on LIBERO-10 and 39.1% on ManiSkill, with lower SR in both settings.

2.6 Real-World Setup and Evaluation

Hardware and observations. We use Franka R3 robotic arms equipped with two-finger grippers. A calibrated RealSense D455 camera provides the third-person view, and a ZED-M camera provides the wrist view. The policy operates at 15 Hz and outputs absolute end-effector commands.

Policies and training. We train task-specific Diffusion Policy (DP) baselines and Inpainting Policies (IP) on the same demonstration datasets. Both use frozen, pretrained DINOv2 ViT-S/14 encoders for dual-camera RGB observations, together with proprioceptive inputs and a U-Net denoiser with channel dimensions of 256, 512, and 1024. At inference time, IP additionally receives the user-provided action references through a 32-step MAP window. DP serves as the base policy for observation-to-action generation.

Human-in-the-loop control. The Robotic Spatiotemporal Editing system accepts a variable number of user-provided references. Each reference specifies a reference type, the corresponding action values, and a normalized progress value $p' \in [0, 1]$ indicating the task stage at which the constraint should apply. The system converts these inputs into visibility masks and partially specified action tokens in the Masked Action Space (MAP) interface. At inference time, a progress model estimates the current task-progress anchor and selects the corresponding MAP window. IP conditions on this window, visual observations, and proprioceptive state to reconstruct the unspecified action dimensions and predict a complete action chunk for execution. Here, MAP denotes the reference interface, and IP denotes the learned policy used by the interactive system.

Tasks and reference configurations. We evaluate whether references allow IP to satisfy additional spatial and orientational objectives while completing the underlying manipulation task. Across the real-world evaluation, the system uses heterogeneous reference types, including keypoints, flow, poses, and orientations, under both sparse and dense reference configurations. In cup arrangement, the underlying objective is to pick up a cup and place it on a plate; the references additionally control how the cup is grasped and where it is placed. In controlled test-tube pouring, the underlying objective is to move a test tube above a beaker and pour liquid; orientation references specify the tube's tilt angle to regulate the dispensed volume. Figure 6 in the main paper illustrates the reference-guided behaviors and summarizes the reported outcomes.

Evaluation metrics. We report task success rate together with task-specific physical reference errors. For cup arrangement, the physical error is the cup-to-reference-point distance in millimeters; for pouring, it is the tube-tilt error in degrees. These quantities describe task outcomes and are distinct from normalized action-space CE, as discussed in Section 2.3. The rollouts also provide qualitative evidence about motion smoothness and response to the requested references. The per-configuration tables below additionally report normalized action-space CE from the recorded

evaluations. These CE values are dimensionless and should not be interpreted as millimeter distances or tilt angles.

Cup arrangement results. For the real-world results described in the main paper, IP achieves success rates above 93% with three or more reference points, and the cup-to-reference-point distance ranges from 18.2 to $63.7\,\mathrm{mm}$, with a mean of $44.3\,\mathrm{mm}$. With only a single reference at the final progress point, task success falls below 30%. Table 9 reports success rates, normalized CE, and final TCP error across cup-reference configurations. These additional evaluations span different reference configurations and checkpoints; their scope differs from the results summarized in the main paper. IP reaches 100% SR with four 3D reference points at 20k updates and eight 2D reference points at 15k updates. For the 15k 3D configurations, increasing K from 2 to 4 is accompanied by an SR increase from 12.50% to 87.50% and a CE decrease from 0.175191 to 0.063748. The mean final TCP error is $32.60\,\mathrm{mm}$ for the 20k 3D configuration with $K = 4$, and 41.29 and $36.53\,\mathrm{mm}$ for the 15k 3D configurations with $K = 2$ and $K = 4$, respectively. For the 15k 2D configurations, the corresponding errors are 66.29, 56.35, and $59.16\,\mathrm{mm}$ for $K = 4$, 8, and 16, respectively (Table 9). The results support using references across relevant task stages, while success also depends on reference type, placement, and policy checkpoint. The exact SR and CE for the single-point configuration are not available in this table.

Table 9: **Cup arrangement results by reference configuration.** IP denotes Inpainting Policy without RePaint; K is the number of reference points. Checkpoints are identified by their training step, with k denoting 1,000 updates; a dash indicates an unreported or inapplicable value. Positional error is the 3D distance from the last recorded TCP position to the reference endpoint, averaged equally over all rollouts in each configuration, including failures, and indicating the positional error of the cup.

Policy	Reference	K	Checkpoint	SR (%) ↑	Norm. CE↓	Positional error (mm)↓
IP	3D position	4	20k	**100.00**	0.078249	32.60
IP	2D position	16	15k	80.00	0.069302	59.16
IP	3D position	4	15k	87.50	0.063748	36.53
IP	3D position	2	15k	12.50	0.175191	41.29
IP	2D position	4	15k	0.00	0.125435	66.29
IP	2D position	8	15k	**100.00**	0.048606	56.35
IP	3D position	1	15k	—	—	—
DP	None	—	100k	50.00	—	—

Liquid-pouring results. Table 10 shows that IP achieves 93.33% SR with four orientation references and 100% with eight or sixteen references. The reported angle errors range from $7.6867°$ to $7.8133°$, consistent with the below-$8°$ error and mean error of $7.8°$ reported in the main paper. Normalized CE varies non-monotonically with reference count, with the lowest value of 0.090101 at $K = 16$. Thus, task completion, normalized reference error, and physical tilt error capture different aspects of control performance.

Table 10: **Controlled test-tube pouring results by reference count.** All IP rows use the `orientation0.01 / STPM` configuration without RePaint; K is the number of orientation references. DP receives no reference. The DP success labels remain unverified; angle errors are means for the corresponding reference configurations.

Policy	K	Checkpoint	SR (%) ↑	Norm. CE↓	Angle error (°) ↓
IP	4	30k	93.33	0.094315	7.8133
IP	8	30k	**100.00**	0.101605	7.6867
IP	16	30k	**100.00**	0.090101	7.7567
DP	—	50k	43.48	—	—

Aggregate results. Table 11 summarizes the recorded configurations, yielding IP success rates of 77.19% for cup arrangement and 97.78% for pouring, with normalized CE values of 0.086105 and

0.095340, respectively. The corresponding DP success rates are 50% and 43.48%, with the latter subject to the label-verification limitation noted in Table 10. The IP aggregates pool the additional reference configurations and checkpoints reported here; they do not represent the performance of one fixed policy configuration. The high-success results described in the main paper refer to its evaluated reference-guided settings, while these tables provide additional results across configurations.

Table 11: **Aggregate real-world results across recorded reference configurations.** Cup denotes cup arrangement; Water denotes controlled test-tube pouring. IP excludes RePaint. The Water DP success labels remain unverified.

Task	Policy	SR (%) ↑	Norm. CE↓
Cup	IP	**77.19**	0.086105
Cup	DP	50.00	—
Water	IP	**97.78**	0.095340
Water	DP	43.48	—

Practical tips for real-robot deployment. Progress used for window selection should evolve smoothly and monotonically. Discontinuous or backward updates can skip reference points or reactivate constraints from an earlier task stage. A calibrated mapping from predicted progress to reference position, with limits on backward motion and abrupt advances, helps maintain consistent window selection. Progress completion should be distinguished from successful physical task completion.

References should cover the task stages that matter for the requested behavior. For cup arrangement, guidance during grasping and transport can complement a final placement target; for pouring, orientation references can specify the intended progression of tube tilt. Because IP receives a local MAP window, a single late-stage reference may not become visible until execution approaches its associated progress value.

Reference positions and orientations should remain compatible with the training distribution and with one another. Requests far outside the demonstrated range can fail even when they are representable through MAP. IP reconstructs actions using learned behavior, so encoding a constraint in the interface does not by itself guarantee reliable execution of every requested behavior.

3 Additional Method Details

Throughout this section, t indexes policy execution, p indexes reference locations, and k indexes diffusion steps.

3.1 Progress-Guided MAP Alignment

Progress estimation. We use normalized task progress rather than the raw execution timestep to align control references with policy execution. Given the current observation $o_t = (I_t, s_t)$, where I_t denotes camera observations and s_t denotes the robot state, a separately trained progress estimator predicts

$$p'_t = g_\phi(o_t), \qquad p'_t \in [0, 1].$$

Using estimated progress to select references accounts for variations in execution speed and aligns references with the corresponding task stages.

Mapping progress to the reference trajectory. Let $A^{\text{ref}} \in \mathbb{R}^{T_{\text{ref}} \times 7}$ denote the reference action-value array at progress locations p'_p, where $p = 0, \ldots, T_{\text{ref}} - 1$. Each row is an action vector $a_p = (x_p, y_p, z_p, \theta_{x,p}, \theta_{y,p}, \theta_{z,p}, g_p)$, while each scalar entry is a spatiotemporal action token. The binary visibility mask $M \in \{0, 1\}^{T_{\text{ref}} \times 7}$ identifies specified entries, and $V = M \odot A^{\text{ref}}$ contains the masked reference values. The current progress estimate selects the nearest reference index $p_t = \arg\min_p |p'_p - p'_t|$. For uniformly spaced reference progress values $p'_p = p/(T_{\text{ref}} - 1)$ with $T_{\text{ref}} > 1$, this becomes

$$p_t = \tau(p'_t) = \lfloor p'_t (T_{\text{ref}} - 1) \rceil \,. \tag{4}$$

Given h preceding reference locations and j additional future reference locations, we select

$$\mathcal{W}_t = \text{MAP_Window}(p_t) = [a_{p_t-h}, \ldots, a_{p_t}, \ldots, a_{p_t+j}], \tag{5}$$

with indices clipped to $[0, T_{\text{ref}} - 1]$. For out-of-range locations, we repeat the nearest boundary action and its visibility mask, preserving a fixed window length of $h + j + 1$. The forward portion contains $j + 1$ locations, including the current anchor. Thus, a reported forward window length of L_{forward} corresponds to $j = L_{\text{forward}} - 1$ and is distinct from the prediction horizon H.

This formulation avoids directly using normalized progress p'_t as a discrete trajectory index and explicitly defines the correspondence between task progress and the reference sequence.

MAP encoding. Although $\mathcal{W}_t$ is written as an action sequence for clarity, the encoder receives only the corresponding masked values and visibility masks. Let $V_t^{\text{win}} \in \mathbb{R}^{(h+j+1)\times 7}$ and $M_t^{\text{win}} \in \{0,1\}^{(h+j+1)\times 7}$ denote the rows selected from V and M using the same indices and boundary handling as $\mathcal{W}_t$. We provide these representations to a lightweight 2D convolutional encoder, keeping the visibility mask explicit so that an unspecified entry can be distinguished from a specified zero value. The encoder produces a 128-dimensional MAP feature vector, which is concatenated with visual and proprioceptive features to condition the denoising network.

The local window serves two purposes. First, it associates spatial constraints with their corresponding interaction stage. Second, it supplies nearby reference context whose temporal coverage is controlled by h and j. Section 1.2 evaluates the effect of forward window length, and Section 1.5 compares estimated progress with rollout-timestep alignment.

3.2 RePaint Inference

The standard Inpainting Policy (IP) generates a complete action chunk conditioned on observations and the local MAP window. We additionally consider a RePaint-style inference strategy (Lugmayr et al., 2022) that reinserts specified reference values during denoising without additional policy training.

Let $R_t \in \mathbb{R}^{H\times 7}$ contain the clean reference values aligned with the target action chunk, and let $M_t^{\text{chunk}} \in \{0,1\}^{H\times 7}$ identify the specified entries. Only entries selected by M_t^{chunk} are used for replacement; the remaining reference values are not required. The reference values and sampled actions use the same action normalization. Sampling starts from Gaussian noise $A^{(K)} \sim \mathcal{N}(0, I)$, where K is the number of reverse diffusion steps. At each step k, we first obtain a denoised proposal $\widetilde{A}^{(k-1)} = \mathcal{D}_\theta(A^{(k)}, k, C_t)$ and noise the reference values to the corresponding level as $R_t^{(k-1)} = \sqrt{\bar{\alpha}_{k-1}} R_t + \sqrt{1-\bar{\alpha}_{k-1}}\xi_k$. Here, $\mathcal{D}_\theta$ denotes one reverse diffusion step using the noise prediction $\epsilon_\theta(A^{(k)}, k, C_t)$, $\xi_k \sim \mathcal{N}(0, I)$, and C_t contains the observation and local MAP conditioning features. The reverse diffusion step produces an action proposal at noise level $k-1$; the noise prediction itself is not an action sample.

We retain the proposal at unspecified entries and replace the specified entries with the noised reference values:

$$A^{(k-1)} = M_t^{\text{chunk}} \odot R_t^{(k-1)} + (1 - M_t^{\text{chunk}}) \odot \widetilde{A}^{(k-1)}. \tag{6}$$

This replacement is applied at every reverse diffusion step. At the last step, we use clean reference values with $\bar{\alpha}_0 = 1$, so the final sampled action chunk satisfies $M_t^{\text{chunk}} \odot A^{(0)} = M_t^{\text{chunk}} \odot R_t$. This equality concerns the sampled action values and does not guarantee exact reference satisfaction during physical execution.

The learned Inpainting Policy and its RePaint inference variant are evaluated separately in our experiments: the former learns to reconstruct actions through the conditional denoising objective, whereas the latter adds explicit reference replacement during sampling.

3.3 Masked Denoising Objective

Given a clean demonstration action chunk $A^{(0)} \in \mathbb{R}^{H\times 7}$, a diffusion step k, and Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$, we construct

$$A^{(k)} = \sqrt{\bar{\alpha}_k} A^{(0)} + \sqrt{1-\bar{\alpha}_k}\epsilon.$$

Here, $\bar{\alpha}_k$ is the cumulative signal-retention coefficient.

The denoising network predicts $\widehat{\epsilon} = \epsilon_\theta(A^{(k)}, k, C_t)$ from the noisy action chunk and the conditioning features defined in Section 3.6. The visibility mask $M_t^{\text{chunk}} \in \{0,1\}^{H\times 7}$ is aligned with the target action chunk. It is distinct from the full reference mask M and the local conditioning-window mask M_t^{win}, whose temporal lengths are T_{ref} and $h + j + 1$, respectively. We optimize

$$\mathcal{L}_{\text{MAP}} = \mathbb{E}\begin{bmatrix} w_m \left\| M_t^{\text{chunk}} \odot (\epsilon - \widehat{\epsilon}) \right\|_F^2 \\ + w_u \left\| (1 - M_t^{\text{chunk}}) \odot (\epsilon - \widehat{\epsilon}) \right\|_F^2 \end{bmatrix}, \tag{7}$$

where w_m and w_u weight the visible and missing regions, and $\|\cdot\|_F^2$ sums squared errors over the action-chunk entries. The expectation is over training examples, sampled masks, diffusion steps, and Gaussian noise.

The unknown-token objective trains the model to infer actions that are not specified by the reference. The visible-token objective encourages consistency between predicted actions and specified control values. This differs from RePaint inference, where visible values are explicitly overwritten rather than enforced only through the learned denoising model.

3.4 Mask Families for Joint Training

We construct multiple masked-action examples from each demonstration. The training distribution contains three complementary mask families: structured masks (α), no-reference masks (β), and random masks (γ).

α: Structured control masks. The structured family represents control references that naturally arise from external planning or human specification. It includes:

- 2D position or trajectory constraints;
- 3D position or trajectory constraints;
- 7D poses comprising Cartesian position, orientation, and gripper state;
- orientation-only constraints;
- gripper-state constraints; and
- sparse keypoints or keyposes.

These masks expose semantically meaningful subsets of the Cartesian action trajectory and teach the policy to interpret different reference modalities within a shared representation. The first three coordinates specify Cartesian position, the next three specify orientation, and the seventh specifies the gripper state. Here, 2D references constrain Cartesian x, y coordinates, while flow references specify position trajectories across reference locations. Image-space references must first be mapped into the action coordinate system. For a reference at progress p', we select $p^\star = \arg\min_p |p'_p - p'|$ and reveal the corresponding action coordinates in row $p^\star$ of M. For example, a 3D point reveals coordinates $\{1, 2, 3\}$, while an orientation reference reveals $\{4, 5, 6\}$.

β: No-reference masks. For this family, the visibility ratio is zero:

$$M = \mathbf{0}.$$

The policy therefore generates actions without an external action reference while retaining its observation conditioning. Including this case trains the policy to operate when action references are unavailable.

γ: Random token masks. Random masks reveal randomly selected scalar action entries across reference locations and action dimensions. Unlike the structured family, these patterns are not tied to a predefined control modality. They expose the model to a substantially larger space of partial action specifications and encourage token-level reconstruction beyond the structured patterns seen during training.

We sample a mask family and then a mask conditioned on that family:

$$c \sim \rho(c), \quad c \in \{\alpha, \beta, \gamma\}, \qquad M \sim p(M \mid c),$$

and apply M to the demonstration action trajectory to construct the reference condition for diffusion training. The distribution $\rho(c)$ controls the relative frequency of structured, no-reference, and random-mask examples. For LIBERO-10, the two-stage schedule first uses random masks and then continues with a mixture of structured masks spanning different reference modalities and sparsity levels. The training settings and schedule comparison are described in Sections 2.2 and 1.4, respectively.

3.5 Generalization to Unseen Mask Compositions

Because all control signals are represented as visible entries in the same $T_{\mathrm{ref}} \times 7$ reference action space, compatible reference modalities can be combined by taking the union of their visible tokens. For two masks M_1 and M_2 defined on the same reference progress grid, their composition is

$$M_{\mathrm{mix}} = M_1 \vee M_2, \tag{8}$$

with the corresponding visible action entries assembled from the two references. This composition assumes that any overlapping specified entries agree.

This representation does not introduce a modality-specific input head or control interface. Consequently, the policy can accept a mask composition that does not appear explicitly in the training set. Our experiments evaluate its ability to follow such compositions using `Mix0`, a mixed-mask pattern excluded from training, without additional policy fine-tuning. The construction of `Mix0` is defined in Section 2.4.

3.6 Conditioning Architecture

The denoising model is conditioned on three information sources:

$$C_t = \left[\phi_{\mathrm{vis}}(I_t), \phi_{\mathrm{state}}(s_t), \phi_{\mathrm{MAP}}\left(M_t^{\mathrm{win}}, V_t^{\mathrm{win}}\right)\right]. \tag{9}$$

Here, ϕ_{vis} extracts visual features, ϕ_{state} encodes robot proprioception, and ϕ_{MAP} embeds the local masked-action window defined in Section 3.1. The MAP encoder uses 2D convolutions followed by pooling and projection to produce a 128-dimensional feature vector. The brackets denote concatenation of this vector with the visual and proprioceptive features. The resulting condition C_t is supplied to $\epsilon_\theta(A^{(k)}, k, C_t)$ during action denoising. The encoder comparison is reported in Section 1.1.

The formulation is agnostic to the specific diffusion backbone. In our experiments, we instantiate it using either a Diffusion Transformer or a U-Net-based Diffusion Policy depending on the benchmark. See Section 2.2 for policy architectures and training settings.